



## Definitions

# Definitions

## Definition

*Concurrency* is an abstraction for the programmer, allowing programs to be structured as multiple **threads of control**, called *processes*. These processes may communicate in various ways.

**Example Applications:** Servers, OS Kernels, GUI applications.

# Definitions

## Definition

*Concurrency* is an abstraction for the programmer, allowing programs to be structured as multiple **threads of control**, called *processes*. These processes may communicate in various ways.

**Example Applications:** Servers, OS Kernels, GUI applications.

## Anti-definition

Concurrency is **not parallelism**, which is a means to exploit multiprocessing hardware in order to improve performance.

## Sequential vs Concurrent

We could consider a *sequential* program as a *sequence* (or *total order*) of *actions*:



## Sequential vs Concurrent

We could consider a *sequential* program as a *sequence* (or *total order*) of *actions*:



The ordering here is “happens before”. For example, processor instructions:

LD R0,X → LDI R1,5 → ADD R0,R1 → ST X,R0

## Sequential vs Concurrent

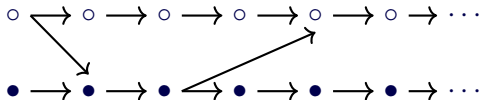
We could consider a *sequential* program as a *sequence* (or *total order*) of *actions*:



The ordering here is “happens before”. For example, processor instructions:

LD R0,X → LDI R1,5 → ADD R0,R1 → ST X,R0

A concurrent program is not a total order but a *partial order*.



This means that there are now multiple possible *interleavings* of these actions — our program is *non-deterministic* where the interleaving is selected by the scheduler.

# Concurrent Programs

Consider the following concurrent processes, sharing a variable  $n$ .

| <b>var <math>n := 0</math></b> |                            |                            |
|--------------------------------|----------------------------|----------------------------|
| $p_1: \text{var } x := n;$     | $q_1: \text{var } y := n;$ | $r_1: \text{var } z := n;$ |
| $p_2: n := x + 1;$             | $q_2: n := y - 1;$         | $r_2: n := z + 1;$         |

## Question

What are the possible returned values?



# A Sobering Realisation

How many scenarios are there for a program with  $n$  processes consisting of  $m$  steps each?

## A Sobering Realisation

How many scenarios are there for a program with  $n$  processes consisting of  $m$  steps each?

|         | $n = 2$ | 3          | 4          | 5          | 6          |
|---------|---------|------------|------------|------------|------------|
| $m = 2$ | 6       | 90         | 2520       | 113400     | $2^{22.8}$ |
| 3       | 20      | 1680       | $2^{18.4}$ | $2^{27.3}$ | $2^{36.9}$ |
| 4       | 70      | 34650      | $2^{25.9}$ | $2^{38.1}$ | $2^{51.5}$ |
| 5       | 252     | $2^{19.5}$ | $2^{33.4}$ | $2^{49.1}$ | $2^{66.2}$ |
| 6       | 924     | $2^{24.0}$ | $2^{41.0}$ | $2^{60.2}$ | $2^{81.1}$ |

## A Sobering Realisation

How many scenarios are there for a program with  $n$  processes consisting of  $m$  steps each?

|         | $n = 2$ | 3          | 4          | 5          | 6          |
|---------|---------|------------|------------|------------|------------|
| $m = 2$ | 6       | 90         | 2520       | 113400     | $2^{22.8}$ |
| 3       | 20      | 1680       | $2^{18.4}$ | $2^{27.3}$ | $2^{36.9}$ |
| 4       | 70      | 34650      | $2^{25.9}$ | $2^{38.1}$ | $2^{51.5}$ |
| 5       | 252     | $2^{19.5}$ | $2^{33.4}$ | $2^{49.1}$ | $2^{66.2}$ |
| 6       | 924     | $2^{24.0}$ | $2^{41.0}$ | $2^{60.2}$ | $2^{81.1}$ |

$$\frac{(nm)!}{m!^n}$$

# Volatile Variables

| <b>var</b> $y, z := 0, 0$ |                    |
|---------------------------|--------------------|
| $p_1$ : <b>var</b> $x$ ;  | $q_1$ : $y := 1$ ; |
| $p_2$ : $x := y + z$ ;    | $q_2$ : $z := 2$ ; |

## Question

What are the possible final values of  $x$ ?

## Volatile Variables

| <b>var</b> $y, z := 0, 0$ |                    |
|---------------------------|--------------------|
| $p_1$ : <b>var</b> $x$ ;  | $q_1$ : $y := 1$ ; |
| $p_2$ : $x := y + z$ ;    | $q_2$ : $z := 2$ ; |

### Question

What are the possible final values of  $x$ ?

What about  $x = 2$ ? Is that possible?

## Volatile Variables

| <b>var</b> $y, z := 0, 0$ |                    |
|---------------------------|--------------------|
| $p_1$ : <b>var</b> $x$ ;  | $q_1$ : $y := 1$ ; |
| $p_2$ : $x := y + z$ ;    | $q_2$ : $z := 2$ ; |

### Question

What are the possible final values of  $x$ ?

What about  $x = 2$ ? Is that possible?

It **is** possible, as we cannot guarantee that the statement  $p_2$  is executed **atomically** — that is, as one step.

## Volatile Variables

| var $y, z := 0, 0$       |                    |
|--------------------------|--------------------|
| $p_1$ : <b>var</b> $x$ ; | $q_1$ : $y := 1$ ; |
| $p_2$ : $x := y + z$ ;   | $q_2$ : $z := 2$ ; |

### Question

What are the possible final values of  $x$ ?

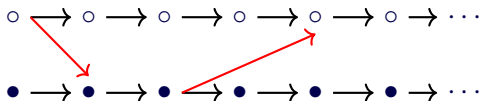
What about  $x = 2$ ? Is that possible?

It **is** possible, as we cannot guarantee that the statement  $p_2$  is executed **atomically** — that is, as one step.

Typically, we require that each statement only accesses (reads from or writes to) at most **one** shared variable at a time. Otherwise, we cannot guarantee that each statement is one atomic step. This is called the **limited critical reference** restriction.

# Synchronisation

In order to reduce the number of possible interleavings, we must allow processes to synchronise their behaviour, ensuring more orderings (and thus fewer interleavings).



The red arrows are synchronisations.



# Atomicity

The basic unit of synchronisation we would like to implement is to group multiple steps into one atomic step, called a *critical section*.

# Atomicity

The basic unit of synchronisation we would like to implement is to group multiple steps into one atomic step, called a *critical section*. A sketch of the problem can be outlined as follows:

|                                                                                                                            |                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>forever do</b><br><i>non-critical section</i><br><i>pre-protocol</i><br><b>critical section</b><br><i>post-protocol</i> | <b>forever do</b><br><i>non-critical section</i><br><i>pre-protocol</i><br><b>critical section</b><br><i>post-protocol</i> |
|----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|

## Atomicity

The basic unit of synchronisation we would like to implement is to group multiple steps into one atomic step, called a *critical section*. A sketch of the problem can be outlined as follows:

|                                                                                                                            |                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>forever do</b><br><i>non-critical section</i><br><i>pre-protocol</i><br><b>critical section</b><br><i>post-protocol</i> | <b>forever do</b><br><i>non-critical section</i><br><i>pre-protocol</i><br><b>critical section</b><br><i>post-protocol</i> |
|----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|

The non-critical section models the possibility that a process may do something else. It can take any amount of time (even infinite).

# Atomicity

The basic unit of synchronisation we would like to implement is to group multiple steps into one atomic step, called a *critical section*. A sketch of the problem can be outlined as follows:

|                                                                                                                            |                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>forever do</b><br><i>non-critical section</i><br><i>pre-protocol</i><br><b>critical section</b><br><i>post-protocol</i> | <b>forever do</b><br><i>non-critical section</i><br><i>pre-protocol</i><br><b>critical section</b><br><i>post-protocol</i> |
|----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|

The non-critical section models the possibility that a process may do something else. It can take any amount of time (even infinite). Our task is to find a pre- and post-protocol such that certain *atomicity properties* are satisfied.

## Desiderata

We want to ensure two main properties:

- **Mutual Exclusion** No two processes are in their critical section at the same time.
- **Eventual Entry** (or *starvation-freedom*) Once it enters its pre-protocol, a process will eventually be able to execute its critical section.

### Question

Which is safety and which is liveness?

## Desiderata

We want to ensure two main properties:

- **Mutual Exclusion** No two processes are in their critical section at the same time.
- **Eventual Entry** (or *starvation-freedom*) Once it enters its pre-protocol, a process will eventually be able to execute its critical section.

### Question

Which is safety and which is liveness?

Mutex is safety, Eventual Entry is liveness.

## First Attempt

We can implement **await** using primitive machine instructions or OS syscalls, or even using a busy-waiting loop.

| <b>var turn := 1</b>                         |                                              |
|----------------------------------------------|----------------------------------------------|
| <b>forever do</b>                            | <b>forever do</b>                            |
| p <sub>1</sub> <i>non-critical section</i>   | q <sub>1</sub> <i>non-critical section</i>   |
| p <sub>2</sub> <b>await</b> <i>turn = 1;</i> | q <sub>2</sub> <b>await</b> <i>turn = 2;</i> |
| p <sub>3</sub> <b>critical section</b>       | q <sub>3</sub> <b>critical section</b>       |
| p <sub>4</sub> <i>turn := 2</i>              | q <sub>4</sub> <i>turn := 1</i>              |

### Question

Mutual Exclusion?

## First Attempt

We can implement **await** using primitive machine instructions or OS syscalls, or even using a busy-waiting loop.

| <b>var</b> <i>turn</i> := 1                         |                                                     |
|-----------------------------------------------------|-----------------------------------------------------|
| <b>forever do</b>                                   | <b>forever do</b>                                   |
| <i>p</i> <sub>1</sub> <i>non-critical section</i>   | <i>q</i> <sub>1</sub> <i>non-critical section</i>   |
| <i>p</i> <sub>2</sub> <b>await</b> <i>turn</i> = 1; | <i>q</i> <sub>2</sub> <b>await</b> <i>turn</i> = 2; |
| <i>p</i> <sub>3</sub> <b>critical section</b>       | <i>q</i> <sub>3</sub> <b>critical section</b>       |
| <i>p</i> <sub>4</sub> <i>turn</i> := 2              | <i>q</i> <sub>4</sub> <i>turn</i> := 1              |

### Question

Mutual Exclusion? Yup!



## First Attempt

We can implement **await** using primitive machine instructions or OS syscalls, or even using a busy-waiting loop.

| <b>var</b> <i>turn</i> := 1                         |                                                     |
|-----------------------------------------------------|-----------------------------------------------------|
| <b>forever do</b>                                   | <b>forever do</b>                                   |
| <i>p</i> <sub>1</sub> <i>non-critical section</i>   | <i>q</i> <sub>1</sub> <i>non-critical section</i>   |
| <i>p</i> <sub>2</sub> <b>await</b> <i>turn</i> = 1; | <i>q</i> <sub>2</sub> <b>await</b> <i>turn</i> = 2; |
| <i>p</i> <sub>3</sub> <b>critical section</b>       | <i>q</i> <sub>3</sub> <b>critical section</b>       |
| <i>p</i> <sub>4</sub> <i>turn</i> := 2              | <i>q</i> <sub>4</sub> <i>turn</i> := 1              |

### Question

Mutual Exclusion? Yup!  
Eventual Entry?

## First Attempt

We can implement **await** using primitive machine instructions or OS syscalls, or even using a busy-waiting loop.

| <b>var</b> <i>turn</i> := 1                         |                                                     |
|-----------------------------------------------------|-----------------------------------------------------|
| <b>forever do</b>                                   | <b>forever do</b>                                   |
| <i>p</i> <sub>1</sub> <i>non-critical section</i>   | <i>q</i> <sub>1</sub> <i>non-critical section</i>   |
| <i>p</i> <sub>2</sub> <b>await</b> <i>turn</i> = 1; | <i>q</i> <sub>2</sub> <b>await</b> <i>turn</i> = 2; |
| <i>p</i> <sub>3</sub> <b>critical section</b>       | <i>q</i> <sub>3</sub> <b>critical section</b>       |
| <i>p</i> <sub>4</sub> <i>turn</i> := 2              | <i>q</i> <sub>4</sub> <i>turn</i> := 1              |

### Question

Mutual Exclusion? Yup!  
Eventual Entry? Nope!

## First Attempt

We can implement **await** using primitive machine instructions or OS syscalls, or even using a busy-waiting loop.

| <b>var</b> <i>turn</i> := 1                         |                                                     |
|-----------------------------------------------------|-----------------------------------------------------|
| <b>forever do</b>                                   | <b>forever do</b>                                   |
| <i>p</i> <sub>1</sub> <i>non-critical section</i>   | <i>q</i> <sub>1</sub> <i>non-critical section</i>   |
| <i>p</i> <sub>2</sub> <b>await</b> <i>turn</i> = 1; | <i>q</i> <sub>2</sub> <b>await</b> <i>turn</i> = 2; |
| <i>p</i> <sub>3</sub> <b>critical section</b>       | <i>q</i> <sub>3</sub> <b>critical section</b>       |
| <i>p</i> <sub>4</sub> <i>turn</i> := 2              | <i>q</i> <sub>4</sub> <i>turn</i> := 1              |

### Question

Mutual Exclusion? Yup!

Eventual Entry? Nope! What if *q*<sub>1</sub> never finishes?

## Second Attempt

| <b>var</b> <i>wantp, wantq</i> := False, False                                                                                                                                                                                                 |                                                                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>forever do</b><br>p <sub>1</sub> <i>non-critical section</i><br>p <sub>2</sub> <b>await</b> <i>wantq</i> = False;<br>p <sub>3</sub> <i>wantp</i> := True;<br>p <sub>4</sub> <b>critical section</b><br>p <sub>7</sub> <i>wantp</i> := False | <b>forever do</b><br>q <sub>1</sub> <i>non-critical section</i><br>q <sub>2</sub> <b>await</b> <i>wantp</i> = False;<br>q <sub>3</sub> <i>wantq</i> := True;<br>q <sub>4</sub> <b>critical section</b><br>q <sub>7</sub> <i>wantq</i> := False |

## Second Attempt

| <b>var</b> <i>wantp, wantq</i> := False, False                                                                                                                                                                                                 |                                                                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>forever do</b><br>p <sub>1</sub> <i>non-critical section</i><br>p <sub>2</sub> <b>await</b> <i>wantq</i> = False;<br>p <sub>3</sub> <i>wantp</i> := True;<br>p <sub>4</sub> <b>critical section</b><br>p <sub>7</sub> <i>wantp</i> := False | <b>forever do</b><br>q <sub>1</sub> <i>non-critical section</i><br>q <sub>2</sub> <b>await</b> <i>wantp</i> = False;<br>q <sub>3</sub> <i>wantq</i> := True;<br>q <sub>4</sub> <b>critical section</b><br>q <sub>7</sub> <i>wantq</i> := False |

**Mutual exclusion** is violated if they execute in lock-step (i.e. p<sub>1</sub>q<sub>1</sub>p<sub>2</sub>q<sub>2</sub>p<sub>3</sub>q<sub>3</sub> etc.)

## Third Attempt

| <b>var</b> <i>wantp, wantq</i> := False, False                                                                                                                                                                                                 |                                                                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>forever do</b><br>p <sub>1</sub> <i>non-critical section</i><br>p <sub>2</sub> <i>wantp</i> := True;<br>p <sub>3</sub> <b>await</b> <i>wantq</i> = False;<br>p <sub>4</sub> <b>critical section</b><br>p <sub>7</sub> <i>wantp</i> := False | <b>forever do</b><br>q <sub>1</sub> <i>non-critical section</i><br>q <sub>2</sub> <i>wantq</i> := True;<br>q <sub>3</sub> <b>await</b> <i>wantp</i> = False;<br>q <sub>4</sub> <b>critical section</b><br>q <sub>7</sub> <i>wantq</i> := False |

## Third Attempt

| <b>var</b> <i>wantp, wantq</i> := False, False                                                                                                                                                                                                 |                                                                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>forever do</b><br>p <sub>1</sub> <i>non-critical section</i><br>p <sub>2</sub> <i>wantp</i> := True;<br>p <sub>3</sub> <b>await</b> <i>wantq</i> = False;<br>p <sub>4</sub> <b>critical section</b><br>p <sub>7</sub> <i>wantp</i> := False | <b>forever do</b><br>q <sub>1</sub> <i>non-critical section</i><br>q <sub>2</sub> <i>wantq</i> := True;<br>q <sub>3</sub> <b>await</b> <i>wantp</i> = False;<br>q <sub>4</sub> <b>critical section</b><br>q <sub>7</sub> <i>wantq</i> := False |

Now we have a **stuck state** (or **deadlock**) if they proceed in lock step, so this violates **eventual entry** also.

## Fourth Attempt

| <b>var</b> <i>wantp, wantq</i> := False, False                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>forever do</b><br>p <sub>1</sub> <i>non-critical section</i><br>p <sub>2</sub> <i>wantp</i> := True;<br>p <sub>3</sub> <b>while</b> <i>wantq</i> <b>do</b><br>p <sub>4</sub> <i>wantp</i> := False;<br>p <sub>5</sub> <i>wantp</i> := True<br><b>od</b><br>p <sub>6</sub> <b>critical section</b><br>p <sub>7</sub> <i>wantp</i> := False | <b>forever do</b><br>q <sub>1</sub> <i>non-critical section</i><br>q <sub>2</sub> <i>wantq</i> := True;<br>q <sub>3</sub> <b>while</b> <i>wantp</i> <b>do</b><br>q <sub>4</sub> <i>wantq</i> := False;<br>q <sub>5</sub> <i>wantq</i> := True<br><b>od</b><br>q <sub>6</sub> <b>critical section</b><br>q <sub>7</sub> <i>wantq</i> := False |



## Fourth Attempt

| <b>var</b> <i>wantp, wantq</i> := False, False     |                                                    |
|----------------------------------------------------|----------------------------------------------------|
| <b>forever do</b>                                  | <b>forever do</b>                                  |
| p <sub>1</sub> <i>non-critical section</i>         | q <sub>1</sub> <i>non-critical section</i>         |
| p <sub>2</sub> <i>wantp</i> := True;               | q <sub>2</sub> <i>wantq</i> := True;               |
| p <sub>3</sub> <b>while</b> <i>wantq</i> <b>do</b> | q <sub>3</sub> <b>while</b> <i>wantp</i> <b>do</b> |
| p <sub>4</sub> <i>wantp</i> := False;              | q <sub>4</sub> <i>wantq</i> := False;              |
| p <sub>5</sub> <i>wantp</i> := True                | q <sub>5</sub> <i>wantq</i> := True                |
| <b>od</b>                                          | <b>od</b>                                          |
| p <sub>6</sub> <b>critical section</b>             | q <sub>6</sub> <b>critical section</b>             |
| p <sub>7</sub> <i>wantp</i> := False               | q <sub>7</sub> <i>wantq</i> := False               |

We have replaced the **deadlock** with **live lock** (looping) if they continuously proceed in lock-step. Still potentially violates eventual entry.

## Fifth Attempt

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>var</b> <i>wantp</i> , <i>wantq</i> := False, False<br><b>var</b> <i>turn</i> := 1                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>forever do</b><br>p <sub>1</sub> <i>non-critical section</i><br>p <sub>2</sub> <i>wantp</i> = True;<br>p <sub>3</sub> <b>while</b> <i>wantq</i> <b>do</b><br>p <sub>4</sub> <b>if</b> <i>turn</i> = 2 <b>then</b><br>p <sub>5</sub> <i>wantp</i> := False;<br>p <sub>6</sub> <b>await</b> <i>turn</i> = 1;<br>p <sub>7</sub> <i>wantp</i> := True<br><b>fi</b><br><b>od</b><br>p <sub>8</sub> <b>critical section</b><br>p <sub>9</sub> <i>turn</i> := 2<br>p <sub>10</sub> <i>wantp</i> := False | <b>forever do</b><br>q <sub>1</sub> <i>non-critical section</i><br>q <sub>2</sub> <i>wantq</i> = True;<br>q <sub>3</sub> <b>while</b> <i>wantp</i> <b>do</b><br>q <sub>4</sub> <b>if</b> <i>turn</i> = 1 <b>then</b><br>q <sub>5</sub> <i>wantq</i> := False;<br>q <sub>6</sub> <b>await</b> <i>turn</i> = 2;<br>q <sub>7</sub> <i>wantq</i> := True<br><b>fi</b><br><b>od</b><br>q <sub>8</sub> <b>critical section</b><br>q <sub>9</sub> <i>turn</i> := 1<br>q <sub>10</sub> <i>wantq</i> := False |

## Reviewing this attempt

The fifth attempt (**Dekker's algorithm**) works well, unless the scheduler pathologically tries to run the loop at  $q_3 \cdots q_7$  when  $turn = 2$  over and over rather than run the process  $p$  (or vice versa).

What would we need to assume to prevent this?

## Reviewing this attempt

The fifth attempt (**Dekker's algorithm**) works well, unless the scheduler pathologically tries to run the loop at  $q_3 \cdots q_7$  when  $turn = 2$  over and over rather than run the process  $p$  (or vice versa).

What would we need to assume to prevent this?

### Fairness

The **fairness assumption** means that if a process **can** always make a move, it will **eventually** be scheduled to make that move.

With this assumption, Dekker's algorithm is correct.

## Machine Instructions

There exists algorithms to generalise this to any number of processes (**Peterson's algorithm**), but they're outside the scope of this course.

What about if we had a single **machine instruction** to swap two values **atomically**, XC?

## Machine Instructions

There exists algorithms to generalise this to any number of processes (**Peterson's algorithm**), but they're outside the scope of this course.

What about if we had a single **machine instruction** to swap two values **atomically**, XC?

| <b>var common := 1</b>                                                                                                                                                                                                        |                                                                                                                                                                                                                                |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>var tp := 0</b><br><b>forever do</b><br>p1 <i>non-critical section</i><br><b>repeat</b><br>p2      XC( <i>tp, common</i> )<br>p3 <b>until</b> <i>tp</i> = 1<br>p4 <b>critical section</b><br>p5    XC( <i>tp, common</i> ) | <b>var tq := 0</b><br><b>forever do</b><br>q1 <i>non-critical section</i><br><b>repeat</b><br>q2      XC( <i>tq, common</i> );<br>q3 <b>until</b> <i>tq</i> = 1<br>q4 <b>critical section</b><br>q7    XC( <i>tq, common</i> ) |

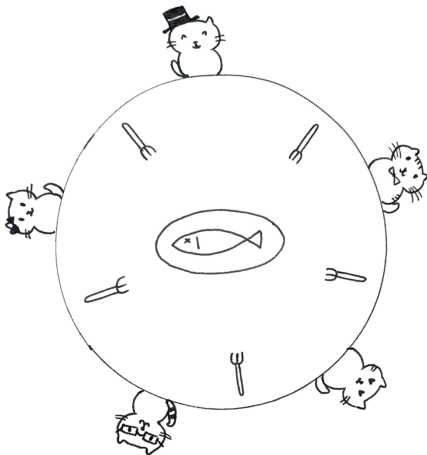
## Locks

The variable *common* is called a *lock*. A lock is the most common means of concurrency control in a programming language implementation. Typically it is abstracted into an abstract data type, with two operations:

- *Taking* the lock — the first exchange (step  $p_2/q_2$ )
- *Releasing* the lock — the second exchange (step  $p_5/q_5$ )

| var lock                             |                                       |
|--------------------------------------|---------------------------------------|
| <b>forever do</b>                    | <b>forever do</b>                     |
| $p_1$ <i>non-critical section</i>    | $q_1$ <i>non-critical section</i>     |
| $p_2$ <b>take</b> ( <i>lock</i> )    | $q_2$ <b>take</b> ( <i>lock</i> );    |
| $p_3$ <b>critical section</b>        | $q_3$ <b>critical section</b>         |
| $p_4$ <b>release</b> ( <i>lock</i> ) | $q_4$ <b>release</b> ( <i>lock</i> ); |

# Dining Philosophers



Five philosophers sit around a dining table with five plates and five forks, all laid out evenly. For whatever reason, philosophers need **two** forks to eat<sup>a</sup>. Philosophers alternate between eating and thinking.

---

<sup>a</sup>More convincing with chopsticks.



## Looks like Critical Sections

**forever do**

*think*

*pre-protocol*

*eat*

*post-protocol*

## Looks like Critical Sections

```
forever do  
  think  
  pre-protocol  
  eat  
  post-protocol
```

For philosopher  $i \in 0 \dots 4$ :

$f_0, f_1, f_2, f_3, f_4$

```
forever do  
  think  
  take( $f_i$ )  
  take( $f_{(i+1) \bmod 5}$ )  
  eat  
  release( $f_i$ )  
  release( $f_{(i+1) \bmod 5}$ )
```

## Looks like Critical Sections

**forever do**

*think*

*pre-protocol*

*eat*

*post-protocol*

For philosopher  $i \in 0 \dots 4$ :

$f_0, f_1, f_2, f_3, f_4$

**forever do**

*think*

**take**( $f_i$ )

**take**( $f_{(i+1) \bmod 5}$ )

*eat*

**release**( $f_i$ )

**release**( $f_{(i+1) \bmod 5}$ )

**Deadlock** is possible (consider lockstep).

## Fixing the Issue

| $f_0, f_1, f_2, f_3, f_4$                                                                                                                                                             |                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Philosophers 0...3                                                                                                                                                                    | Philosopher 4                                                                                                                                             |
| <b>forever do</b><br><i>think</i><br><b>take</b> ( $f_i$ )<br><b>take</b> ( $f_{(i+1) \bmod 5}$ )<br><i>eat</i><br><b>release</b> ( $f_i$ )<br><b>release</b> ( $f_{(i+1) \bmod 5}$ ) | <b>forever do</b><br><i>think</i><br><b>take</b> ( $f_0$ )<br><b>take</b> ( $f_4$ )<br><i>eat</i><br><b>release</b> ( $f_0$ )<br><b>release</b> ( $f_4$ ) |

## Fixing the Issue

| $f_0, f_1, f_2, f_3, f_4$                                                                                                                                                             |                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Philosophers 0...3                                                                                                                                                                    | Philosopher 4                                                                                                                                             |
| <b>forever do</b><br><i>think</i><br><b>take</b> ( $f_i$ )<br><b>take</b> ( $f_{(i+1) \bmod 5}$ )<br><i>eat</i><br><b>release</b> ( $f_i$ )<br><b>release</b> ( $f_{(i+1) \bmod 5}$ ) | <b>forever do</b><br><i>think</i><br><b>take</b> ( $f_0$ )<br><b>take</b> ( $f_4$ )<br><i>eat</i><br><b>release</b> ( $f_0$ )<br><b>release</b> ( $f_4$ ) |

We have to enforce a **global ordering** of locks.